Compiling Facts into Applications

SAMUEL LIPPERT, Drivly Inc, USA

A single set of FORML 2 declarations can define a database schema, constraint rules, state machine workflows, and a REST API with hypermedia navigation, producing a complete application from domain facts alone. We formalize this as AREST, a compositional architecture unifying Backus's FFP/AST [1] with Fielding's REST [4] via Halpin's ORM 2 [5]. Each declaration is an FFP object; the representation function ρ maps it to an executable; the persistent data is Codd's named set [2]; state machine transitions produce HATEOAS links that expose every valid action at every state. Because the API surface is a complete projection of the domain model, an LLM can reason about the system from the API alone. We prove five properties: grammar unambiguity, specification equivalence, completeness of state transfer, HATEOAS as projection, and derivability.

CCS Concepts: • **Software and its engineering** → **Formal language definitions**; **Domain specific languages**; • **Theory of computation** → *Constraint and logic programming*; • **Information systems** → *RESTful web services.*

Additional Key Words and Phrases: applicative state transfer, REST, formal functional programming, object-role modeling, state machines, HATEOAS, first-order logic, specification equivalence

ACM Reference Format:
Samuel Lippert. 2026. Compiling Facts into Applications. *J. ACM* 0, 0, Article 0 (2026), 8 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Definitions

Definition 1.1 (AREST). An AREST system is a tuple (O, ρ, D, P, sub) where:

- O is the set of FFP objects [1, Sec. 13.2]
- $\rho : O \to (O \to O)$ is the representation function [1, Sec. 13.3.2]
- D is the AST state, a sequence of cells [1, Sec. 14.3]
- $P = \uparrow FILE : D$ is the population, a named set of relations [2, Sec. 1.6] whose elements are ORM 2 elementary facts [5, Ch. 2]
- sub is the subsystem function dispatching inputs to state transitions, producing output as REST representations [4] with HATEOAS links

Three terms carry overloaded abbreviations. *AST* follows Backus's usage (Applicative State Transition, 1978), not the compiler term. *ORM* follows Halpin's usage (Object-Role Modeling, 2008), not the persistence pattern. *FP* follows Backus's usage (Function-level Programming), not the broader tradition.

The primitive atoms of O include Backus's structural functions [1, Sec. 11.2.3]. Sequences represent functional forms via metacomposition [1, Sec. 13.3.2]:

$$(\rho\langle x_1, \ldots, x_n\rangle) : y = (\rho\, x_1) : \langle\langle x_1, \ldots, x_n\rangle, y\rangle \tag{1}$$

Author's Contact Information: Samuel Lippert, samuel@driv.ly, Drivly Inc, Shakopee, Minnesota, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-735X/2026/0-ART0
https://doi.org/XXXXXXX.XXXXXXX

The first element determines which functional form; the remaining elements are its parameters. In AREST, a FORML 2 reading is an object; ρ extracts the constraint or derivation function it represents.

2 Operations

2.1 Domain Primitives

Each combining form [1, Sec. 11.2.4] is an FFP object with a domain interpretation:

Table 1. Backus combining forms as FFP objects with AREST domain interpretations.

FFP Object	Form	Domain
$\langle CONS, s_1, \ldots, s_n \rangle$	Construction	Graph Schema
s_i	Selector	Role
$\langle COMP, f, g \rangle$	Composition	Derivation
$\langle COND, p, f, g \rangle$	Condition	Constraint
$\langle \alpha, f \rangle$	Apply-to-all	Traversal
$\langle /, f \rangle$	Insert	Aggregation
$\langle bu, f, x \rangle$	Binary-to-unary	Query
$\bar{x}$	Constant	Literal

A graph schema $\langle CONS, s_1, \ldots, s_n \rangle$ applied to all roles is a fact; partially applied via *bu* it is a query.

2.2 Relational Algebra

Codd's adequate θ_1 [2, Sec. 2.2] as FFP definitions:

$$\pi_L(R) \equiv \alpha\,[s_{i_1}, \ldots, s_{i_k}] : R \tag{2}$$

$$R * S \equiv Filter(eq \circ [s_{sh}]) \circ distl \tag{3}$$

$$R_{L|_M} S \equiv Filter(p) : R \tag{4}$$

$$R \cdot S \equiv \pi_{1s}(R * S) \tag{5}$$

$$\gamma(R) \equiv Filter(eq \circ [s_1, s_n]) : R \tag{6}$$

2.3 State and Cells

The state D is a sequence of cells $\langle CELL, n, c \rangle$ [1, Sec. 14.3]:

$$\uparrow n : D \rightarrow c \qquad \text{(fetch)} \tag{7}$$

$$\downarrow n : \langle x, D \rangle \rightarrow D' \qquad \text{(store)} \tag{8}$$

FILE is a cell whose contents is P. Each entity is a cell. Cells may contain sub-stores [1, Sec. 14.7]; partitioning is a cell containing a subset of P.

2.4 Subsystem Dispatch

Following [1, Sec. 14.4.2]:

$$
\begin{aligned}
&sub \equiv \\
&\quad is\text{-}chg \circ \uparrow K \longrightarrow [rpt, apl] \circ [\uparrow I, defs]; \\
&\quad is\text{-}qry \circ \uparrow K \longrightarrow [qry \circ [\uparrow I, \uparrow F], defs]; \\
&\quad is\text{-}cmd \circ \uparrow K \longrightarrow chk \circ apl \circ [\uparrow I, defs]; \\
&\quad is\text{-}upd \circ \uparrow K \longrightarrow \\
&\qquad [rpt, \downarrow F \circ [upd, defs]] \circ [\uparrow I, \uparrow F]; \\
&\quad [err \circ [\uparrow K, \uparrow I], defs]
\end{aligned} \tag{9}
$$

where $K = KEY$, $I = INPUT$, $F = FILE$.

2.5 Commands

Each interaction is an AST transition [1, Sec. 14.3.1]:

$$\mu(SYSTEM : x) = \langle o, D' \rangle \tag{10}$$

A Create command:

$$create = emit \circ validate \circ derive \circ resolve \tag{11}$$

A state machine is a fold [1, Sec. 11.2.4]:

$$machine = /\,transition : \langle e_1, \ldots, e_n \rangle \tag{12}$$

$$transition = (p \longrightarrow \bar{s'};\ id) \tag{13}$$

3 Example

An order domain is declared in FORML 2:

```
Order(.OrderId) is an entity type.
Customer(.Name) is an entity type.
Order was placed by Customer.
  Each Order was placed by exactly one
  Customer.
Status 'Draft' is initial in State Machine
  Definition 'Order'.
Transition 'place' is from Status 'Draft'.
  Transition 'place' is to Status 'Placed'.
```

The constraint reading is an FFP object; ρ yields:

$$
\begin{aligned}
(\rho\ reading) = \langle COND,\ &(count \leq 1) \circ Filter(eq \circ [s_1, \bar{o}]), \\
&\bar{\phi},\ violation \rangle : P_{placedBy}
\end{aligned}
$$

`POST /orders {"customer":"acme"}` dispatches as *is-cmd* in (9). The engine applies *create* (11):

$$
\begin{aligned}
P' = P \cup \{&Order(\texttt{ord-1}),\ placedBy(\texttt{ord-1}, \texttt{acme}), \\
&SM(\texttt{ord-1}, \texttt{Draft})\}
\end{aligned}
$$

Response (Theorem 5.4: links from P'):

```
{ "id": "ord-1", "customer": "acme",
  "status": "Draft",
  "_links": {
    "place": {
      "href": "/orders/ord-1/transition",
      "method": "POST" } } }
```

Following "place" applies (12). The new representation contains $\{ship\}$, not $\{place\}$. If a second command assigns a different Customer to the same Order, the UC fires and the command is rejected. The violation is returned as the original reading.

4 Execution Semantics

4.1 Evaluation as β-Reduction

FFP objects denote λ-terms via Backus's correspondence [1, Sec. 12.9]. The representation function ρ yields the executable term. Execution is applicative-order (innermost-first) β-reduction to normal form:

μ evaluates an expression under definitions D by repeatedly replacing its innermost application $(x{:}y)$ with $(\rho\, x){:}y$, where ρ resolves x to a λ-term under environment D and primitives over P [1, Sec. 13.3.1]. Each step is a β-reduction: ρ yields the λ-term; application reduces it. The function $apply(func, x, D)$ in the implementation is this reduction step.

All evaluation in AREST is over finite P with monotonic derivations; reduction terminates for all requests.

4.2 Evaluation Strategy

Three constraints govern execution:

(1) **Reduction order:** applicative (call-by-value). All arguments are reduced before the function is applied.
(2) **Scope:** evaluation is confined to the request-local partition of D. RMAP [5, Ch. 17] determines the partition from the schema's uniqueness constraints.
(3) **Derivation:** forward chaining, monotonic, evaluated to the least fixed point per request. Since *derive* only adds facts over finite P_{local}, the fixed point exists (Knaster-Tarski) and is reached in $\leq |P_{local,max}|$ iterations.

The completeness result (Theorem 5.3) assumes evaluation to the least fixed point under this strategy.

4.3 Boundary Conditions

The system operates over three categories:

(1) **Facts** (P): ORM 2 elementary facts, persisted in cells.
(2) **Computations** (O via ρ): total, deterministic functions over P.
(3) **Inputs** (I): request payload, timestamp, and external signals, supplied as the operand x in $\mu(SYSTEM{:}x)$ [1, Sec. 14.3.1].

Time, randomness, and external effects are classified as follows:

- **Time**: an input fact. "Now" is part of x, not hidden state. Time-dependent queries are constraints on timestamp facts in P.
- **Effects**: the output o in $\langle o, D' \rangle$. The engine computes directives (send email, charge payment); the runtime executes them. Outcomes are facts in D'.

- **Non-determinism**: identity generation is a primitive function producing a fresh atom, or the client provides the ID in I.
- **Partial information**: the world assumption per noun [5, Ch. 7] governs incompleteness. CWA: not stated = false. OWA: not stated = unknown.
- **Identity**: the reference scheme [5, Ch. 5] determines identity before facts are asserted. *resolve* in (11) applies the selector.

AREST governs the derivation of representations from (P, I). Theorem 5.5 establishes that every value in the representation is a ρ-application over P.

4.4 The Application as a Fold

A state machine (12) is a fold over an event stream: each event is an input x; the accumulator is the state D; the fold body is $\mu(SYSTEM : x) = \langle o, D' \rangle$. The application IS this fold. The fold is well-defined only if each μ runs to completion before the next event mutates D. Otherwise two concurrent applications of μ may read the same D, and one silently overwrites the other.

Backus states this directly: "the state cannot change during the computation of system:x" [1, Sec. 14.6(c)]. We make the requirement a definition so that theorems can reference it.

Definition 4.1 (Cell Isolation). For each cell $\langle CELL, n, c \rangle$ in D, at most one μ application that writes to n may be in progress at any time. Concurrent μ applications over disjoint cells are permitted.

RMAP [5, Ch. 17] determines which facts belong to which cell from the schema's uniqueness constraints. Each entity is a cell; its fold is independent of other entities' folds. A single-threaded runtime (e.g. a WASM isolate) satisfies Definition 4.1 trivially. A distributed system requires one writer per cell (actor mailbox, partitioned queue, or single-writer lease).

Theorems 5.3–5.5 hold under Definition 4.1. Without it, Theorem 5.3 fails: two concurrent *create* operations on the same cell may both pass *validate* against stale P and produce a D' that violates a uniqueness constraint.

5 Properties

Theorem 5.1 (Grammar Unambiguity). *Let N be a declared set of object type names and F a declared set of predicate readings, such that no element of N contains a formal item of the grammar as a substring. Then each FORML 2 sentence over (N, F) has exactly one parse.*

Proof. A FORML 2 sentence has the structure:

$$[\mathit{modal}] \quad [\mathit{quant}_1] \quad n_1 \quad r \quad \mathit{quant}_2 \quad n_2 \quad [\mathit{qual}]$$

where $\mathit{modal} \in$ {"It is obligatory that", "It is forbidden that", "It is permitted that", ε}; $\mathit{quant}_1 \in$ {"Each", "For each", "the same", ε}; $n_1, n_2 \in N$; $r \in F$; $\mathit{quant}_2 \in$ {"at most one", "some", "exactly one", "at least k and at most m", "more than one", "no", ε}; $\mathit{qual} \in$ {"who ...", "that ...", ε}.

(1) The modal prefixes are pairwise non-overlapping strings, so *modal* is determined by the sentence prefix.

(2) After *modal*, the quantifier quant_1 is determined: "Each" vs. "For each" vs. "the same" vs. a name from N are distinguishable because no $n \in N$ contains a formal item as a substring (by hypothesis).

(3) n_1 is determined by longest-first matching against N. Since names do not contain formal items, the boundary between n_1 and r is unambiguous.

(4) r is the maximal token sequence between n_1 and quant_2. The quantifier phrases in quant_2 are pairwise distinct strings: {"at most one", "some", "exactly one", "at least", "more than one", "no"}.

None is a prefix of another in this position. Since no $n \in N$ contains these phrases, the boundary between r and $quant_2$ is unambiguous.

(5) n_2 is determined by longest-first matching against N after $quant_2$.

(6) *qual* is determined by "who" or "that" (personal vs. impersonal pronouns per [7, Sec. 1.5]).

Each structural position admits exactly one parse. The constraint type is determined by the pair $(quant_1, quant_2)$, which maps injectively to constraint kinds: (Each, at most one) $\longrightarrow$ UC; (Each, some) $\longrightarrow$ MC; (Each, exactly one) $\longrightarrow$ UC+MC; (Each, at least $k\ldots$) $\longrightarrow$ FC; (the same, more than one) $\longrightarrow$ UC$^-$. No two pairs map to the same kind. □

Theorem 5.2 (Specification Equivalence). *Let $parse : R \longrightarrow \Phi$ and $compile : \Phi \longrightarrow O$ be injective, with R the set of FORML 2 readings, Φ the set of FOL formulas, and O the set of FFP objects. Then:*

$$parse^{-1} \circ compile^{-1} \circ compile \circ parse = id_R \tag{14}$$

and $\rho(compile(parse(r)))$ is the executable for every $r \in R$.

Proof. *parse* is injective by Theorem 5.1: each sentence has one parse, so distinct sentences yield distinct formulas. *compile* is injective: the $(quant_1, quant_2)$ pair determines the constraint kind (Theorem 5.1), which determines the FFP object structure (Condition for UC, null-test for MC, count for FC). Both have left inverses; composing: $parse^{-1} \circ compile^{-1} \circ compile \circ parse = id_R$. Executability: ρ maps the compiled object to a function by [1, Sec. 13.4]. □

Theorem 5.3 (Completeness of State Transfer). *Let $F_S : 2^P \longrightarrow 2^P$ be the monotonic operator that applies all derivation rules in schema S to a population, and let C_S be the set of constraint objects in S. Under Definition 4.1, create (11) applied to input I and state D produces $\langle o, D' \rangle$ such that:*

(1) $P' = P \cup resolve(S, I)$ contains all entity facts entailed by (S, I);

(2) $P'' = lfp(F_S, P')$, the least fixed point of F_S above P';

(3) $V = \{(\rho\, c) : P'' \mid c \in C_S\}$ evaluates every constraint;

(4) o is the REST representation of $(P'', V, links(s))$ and $D' = D[FILE \mapsto P'']$ if V contains no alethic violation, else $D' = D$.

Proof. P is finite. F_S is monotonic (derivation rules only add facts). By Knaster-Tarski, $lfp(F_S, P')$ exists and is unique. Iterative application of F_S reaches it in at most $|P''_{\max}| - |P'|$ steps. *validate* evaluates every $c \in C_S$ against the fixed point; no constraint is skipped. *emit* constructs o from P'', V, and $links(s)$ (Theorem 5.4). The state does not change during evaluation [1, Sec. 14.6(c)]; Definition 4.1 ensures no concurrent write to the same cell. □

Theorem 5.4 (HATEOAS as Projection). *Let $T = \{(s_i, s_j, e_k)\} \subseteq P$ be transition facts and s the current status. Then:*

$$links(s) = \pi_{event}(Filter(p) : T) \tag{15}$$

where $p(t) = (s_{from}(t) = s) \vee anc(s_{from}(t), s)$.

Proof. $T \subseteq P$. *Filter* is restriction (4); π is projection (2). Both are θ_1 operations on the named set. The representation renders P; $links(s)$ is derived from P by θ_1 operations and therefore appears in the representation without external computation. □

Theorem 5.5 (Derivability). *Define the* representation *of an entity e with status s in population P under schema S as:*

$$\begin{aligned} repr(e, s, P, S) = &\{(\rho\, s_i) : f \mid f \in P,\ f \text{ involves } e\} \\ &\cup \{(\rho\, r) : P \mid r \in S_{derivations}\} \\ &\cup \{(\rho\, c) : P \mid c \in S_{constraints}\} \\ &\cup links(s) \end{aligned}$$

Then for every $v \in repr(e, s, P, S)$, there exists $f \in O$ such that $v = (\rho\, f) : P$.

Proof. Each component of *repr* is a ρ-application by construction: selectors extract attributes; derivation rules are FFP objects whose ρ-image computes derived facts; constraints are FFP objects whose ρ-image evaluates to ϕ or a violation; links are θ_1 operations (Theorem 5.4). No value in *repr* is produced by a mechanism outside ρ. □

Remark (Query Adequacy). P is a named set of relations in normal form (ORM 2 elementary facts are 5NF by construction [5, Ch. 12]). Codd shows that $\theta_1 = \{$projection, natural join, restriction, tie$\}$ is adequate for the named set [3]. Equations (2)–(6) express each element of θ_1 as an FFP definition.

Remark (Verbalizability). Each θ_1 primitive has a FORML 2 verbalization: projection as role selection, join as qualified composition, restriction as constraint predicate, tie as self-reference. Halpin and Curland [6] give the automated verbalization algorithm for ORM 2 constraints. Composed θ_1 operations verbalize via FORML 2 qualified readings ("who…", "that…"), which parse unambiguously by Theorem 5.1.

Remark (Algebraic Optimization). The algebraic laws of FP [1, Sec. 12.2] are equations between functional forms. Since FFP objects represent the same forms via ρ [1, Sec. 13.3.2], an FFP object can be transformed to an equivalent object by applying these laws. Ahead-of-time compilation from objects to optimized evaluation trees is one such transformation.

6 Conclusion

AREST composes FFP/AST [1] with REST [4] via ORM 2 [5]. Readings are FFP objects; ρ maps them to executables; the population is Codd's named set [2] stored as Backus's FILE cell; transitions produce hypermedia. The five theorems establish: the grammar is unambiguous (Theorem 5.1); specifications and executables are the same object (Theorem 5.2); state transfer is complete in one μ application under cell isolation (Theorem 5.3); HATEOAS is a θ_1 projection (Theorem 5.4); and every value in the representation is a ρ-application over the population (Theorem 5.5). Query adequacy follows from Codd [3]; verbalizability from Halpin and Curland [6]; algebraic optimization from Backus [1, Sec. 12.2].

Acknowledgments

Claude (Anthropic) was used as a programming assistant during implementation and as a drafting aid during manuscript preparation. All architectural decisions and the composition of FFP/AST with REST were made by the human author.

References

[1] J. Backus. Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs. *Communications of the ACM*, 21(8):613–641, 1978.
[2] E.F. Codd. A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, 13(6):377–387, 1970.
[3] E.F. Codd. Relational Completeness of Data Base Sublanguages. In R. Rustin, ed., *Data Base Systems*, Prentice-Hall, pages 65–98, 1972.

[4] R.T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures.* Doctoral dissertation, University of California, Irvine, 2000.
[5] T. Halpin. *Information Modeling and Relational Databases.* 2nd edition. Morgan Kaufmann, 2008.
[6] T. Halpin and M. Curland. Automated Verbalization for ORM 2. In *OTM Workshops*, pages 1181–1190, 2006.
[7] T. Halpin, M. Curland, and CS445 Class. ORM 2 Constraint Verbalization: Part 1. Technical Report ORM2-02, Neumont University, 2006.